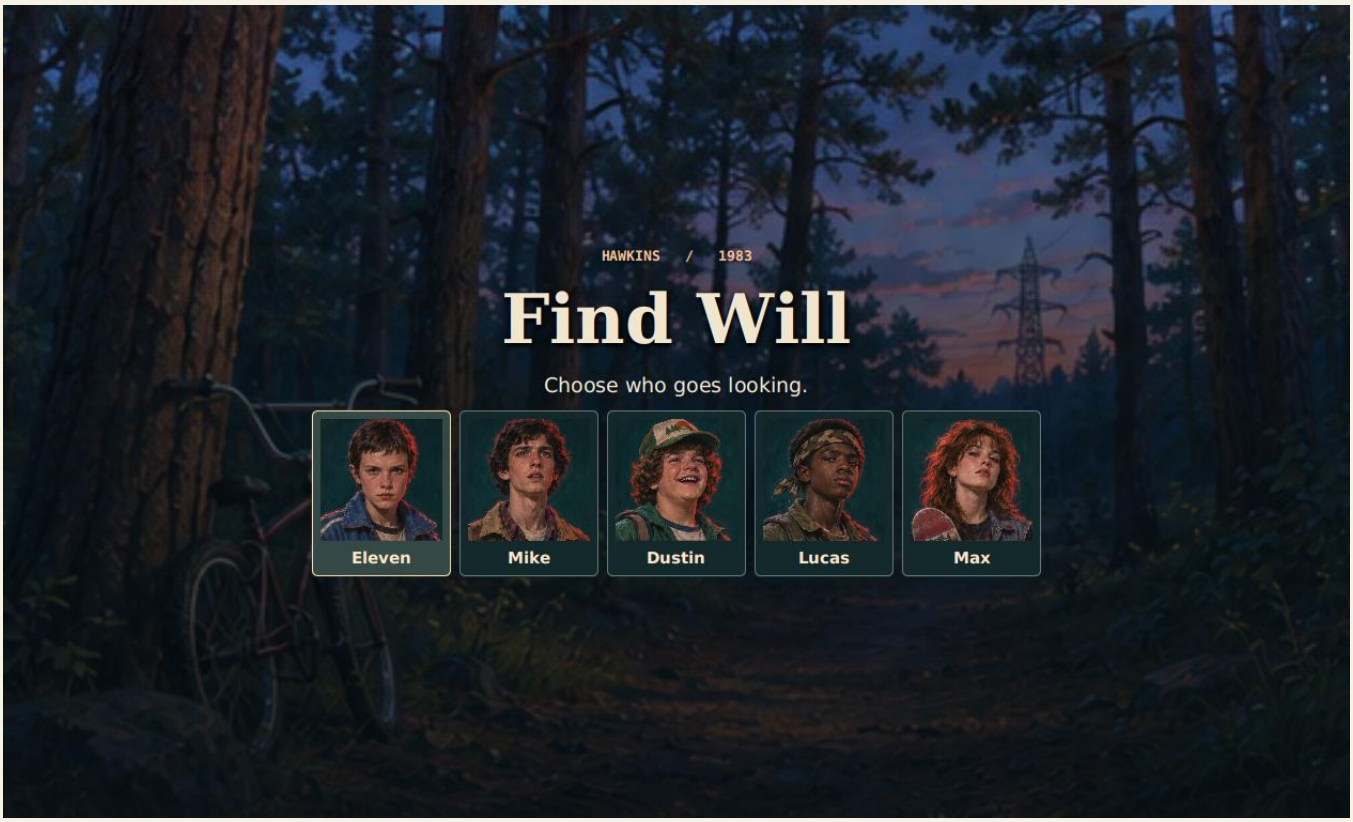
Find Will

Kashtayeva Yasmina / Byerik Farida



Introduction. Five friends can lead one short search for Will Byers. The aim is to show six design patterns through choices a student can demonstrate in a few minutes.

Play. Investigate a radio call or blinking bulbs, take a tool from Will's bicycle, follow the bike light to rescue Will or follow a voice into the Demogorgon's trap. Each has a different reaction to the trail.

Implementation. Java 17, JavaFX controls and media, Java Sound, Maven and JUnit 5. A single in-memory game state tracks the selected friend, location, clue and health.

Six patterns in one search

Builder

RescueMission.Builder assembles the two clue sources, the routes and both world factories. build() validates the setup and freezes the route list.

Factory Method

ClueSource.discover() calls its overridable createClue(). HouseSource returns two discoveries at the house. KnockSource creates a door clue that counts three taps and responds to each.

Strategy

The player selects BridgeRoute or VineRoute at runtime. Both implement RouteStrategy.cross(). The bridge preserves health. The delay in the vines costs Will ten points and clamp the result to at least one.

Abstract Factory

HawkinsWorld and UpsideDownWorld implement WorldFactory. Each supplies a Scenery object and a Soundscape object. The inverted world supplies its own pictures and the one-shot capture cue. The session song loops independently.

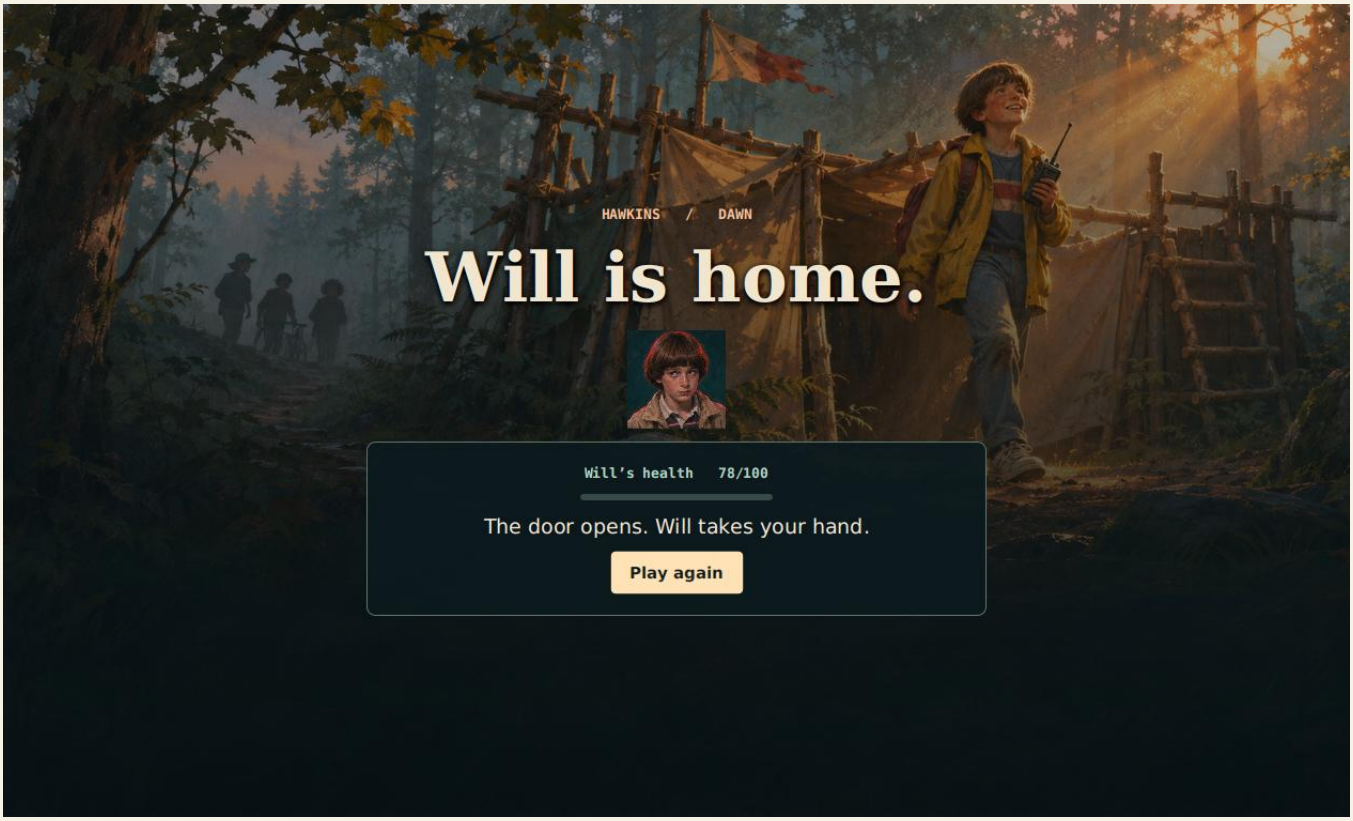
Observer

SearchEvents publishes a SearchView snapshot. GameScreen registers two listeners: one renders the view and the other plays the scene entry cue, when present.

Facade

SearchFacade exposes choose, answer, cross, enterGate and replay. It checks the current step, invokes the appropriate rule and publishes the updated state.

Verification and conclusion



Tests. Maven packaged the application. All 24 JUnit invocations passed. The character test runs each of the five friends through both routes, checks the ending and health, then verifies replay resets the search.

Interface. Two headless JavaFX playthroughs used the actual buttons. Checks at small and wide window sizes verified that the content stayed centered and each button fit inside the window. The startup audio engine opened the supplied background recording on a virtual Linux sound device.

Conclusion. The same short rescue demonstrates all six patterns. Two equally valid leads give different evidence; the chosen bike tool tests that evidence before the final fork. Will loses 2 health every 3 seconds in the Upside Down. The fork decides between rescue and capture; a descending sound plays once on capture.

Further work. Rehearse on the presentation laptop to check speaker output and timing. The two-person script and class diagram are included alongside this report.